第1章：R语言概述与环境搭建

1.1 R语言简介与特点

- **R语言定位**：一种面向统计分析、数据可视化与建模的开源语言。
- **核心优势**
  - 天生适合矩阵与向量化计算。
  - 拥有大量包生态（CRAN > 20,000 个包）。
  - 强大的可视化与统计建模能力（ggplot2, caret, tidymodels）。
  - 社区开放、学术资源丰富。
- **典型应用场景**
  - 学术研究
  - 商业数据分析
  - 金融与风险建模
  - 生物信息与医学统计
  - 机器学习与数据科学教育

1.2 R与其他语言的对比

特性	R	Python	SAS	SPSS
开源性	完全开源	开源	❌ 商业软件	❌ 商业软件
主要用途	统计分析与可视化	通用计算、AI	商业与医疗统计	社科分析
学习成本	中等	低到中	高	低
包生态	极为丰富 (CRAN)	极为丰富 (PyPI)	有限	有限
可视化能力	顶级（ggplot2）	强 (matplotlib, seaborn)	一般	一般

建议：

- 入门数据分析用 **R**，
- 需要模型部署或AI扩展时可 **R + Python 混合使用**。

1.3 安装与开发环境

安装步骤

1. 从 CRAN 官方网站 下载最新版本。
2. Windows 用户建议同时安装 **Rtools**（可编译 C/C++ 扩展包）。
3. 安装 IDE：推荐 **RStudio（Posit Desktop）**。

验证安装

```
# 验证R安装
version
# 验证RStudio工作正常
getwd()
```

1.4 R软件包管理与镜像设置

安装包

```
install.packages("tidyverse")  # 安装数据科学套件
library(tidyverse)             # 加载库
```

查看、更新与删除包

```
installed.packages()
update.packages(ask = FALSE)
remove.packages("dplyr")
```

设置镜像（国内推荐）

```
options(repos = c(CRAN = "https://mirrors.tuna.tsinghua.edu.cn/CRAN/"))
```

1.5 工作目录与文件组织

概念：

- **Working Directory（工作路径）** 是R默认的文件定位位置。
  设置与查看：

```
getwd()                        # 查看当前路径
setwd("C:/project/data")       # 修改路径
```

结构推荐：

```
project/
├── data/           # 原始数据
├── scripts/        # R脚本
├── outputs/        # 图形、结果
├── report/         # 报告文件
└── .Rproj          # 工程文件
```

1.6 R脚本、RMarkdown与Quarto

工具	用途	输出形式
`.R` 脚本	编写与运行R代码	控制台输出
`.Rmd` (R Markdown)	文档 + 代码 + 可视化报告	HTML / PDF / Word
`.qmd` (Quarto)	下一代报告系统，可支持Python+R混合	HTML / PDF / Slides

基本示例 —— **R Markdown 报告头部**

```
---
title: "Data Analysis Report"
author: "Your Name"
output: html_document
---
```

嵌入R代码块：

```
```{r}
summary(cars)
plot(cars)
```
```

第2章：R语言基础语法（R Basic Syntax）

2.1 数据类型与对象（Data Types and Objects）

R是一种**动态类型语言（dynamic typing language）**，不需要提前声明变量类型。
常见数据类型包括：

中文名称	英文名称	示例
数值型	Numeric	`x <- 3.14`
整型	Integer	`y <- 10L`
字符型	Character	`name <- "Alice"`
逻辑型	Logical	`flag <- TRUE`

中文名称	英文名称	示例
因子型	Factor	`gender <- factor(c("M", "F", "M"))`
复数型	Complex	`z <- 1 + 2i`

判断与转换函数（Functions for type checking and conversion）：

```
is.numeric(x)       # 判断是否为数值型 (is numeric)
as.character(x)     # 强制转换为字符型 (convert to character)
as.factor(x)        # 转换为因子型 (convert to factor)
```

2.2 向量、矩阵与数组（Vectors, Matrices, and Arrays）

向量（Vector）

向量是R的最基本数据结构，所有数据结构都是由向量构成的。

```
v <- c(1, 2, 3, 4)
length(v)           # 向量长度 (vector length)
v[2]                # 访问第二个元素 (access the 2nd element)
```

矩阵（Matrix）

矩阵是二维的同类型数据结构。

```
m <- matrix(1:9, nrow = 3, ncol = 3)
m[1, 2]             # 访问第一行第二列 (row 1, column 2)
t(m)                # 矩阵转置 (transpose)
```

数组（Array）

数组可以是三维甚至更高维结构。

```
a <- array(1:12, dim = c(2, 3, 2))
```

2.3 数据框与列表（Data Frames and Lists）

数据框（data.frame）

数据框是最常用的二维结构，类似于Excel表格。

```r
df <- data.frame(
  name = c("Alice", "Bob", "Carol"),
  age = c(23, 25, 30),
  gender = c("F", "M", "F")
)
df$age         # 按列名访问 (access by column name)
df[1, "name"]  # 访问指定单元格
```

列表（list）

列表可以包含任意类型的对象。

```r
person <- list(
  name = "Alice",
  age = 23,
  scores = c(85, 90, 92)
)
person$name
```

2.4 基本运算（Basic Operations）

算术运算符（Arithmetic Operators）

运算	符号	示例
加	`+`	`2 + 3`
减	`-`	`4 - 1`
乘	`*`	`3 * 5`
除	`/`	`10 / 2`
乘方	`^`	`2 ^ 3`
取模	`%%`	`10 %% 3`

逻辑运算符（Logical Operators）

运算	符号	示例
等于	`==`	`x == y`
不等于	`!=`	`x != y`
且	`&`	`x > 0 & y > 0`
或	`|`	`x > 0 | y > 0`
非	`!`	`!(x > 0)`

2.5 条件控制与循环（Control Structures）

条件语句（if / else）

```
x <- 5
if (x > 0) {
  print("Positive number")
} else {
  print("Non-positive number")
}
```

for 循环（for loop）

```
for (i in 1:3) {
  print(i)
}
```

while 循环（while loop）

```
count <- 1
while (count <= 3) {
  print(count)
  count <- count + 1
}
```

apply 系列函数（apply family）

函数	中文名称	用途
`apply()`	通用数组操作	对矩阵行或列求值
`lapply()`	列表操作	对列表中每个元素执行函数
`sapply()`	简化输出的 lapply	常用于向量结果
`tapply()`	分组计算	对数据按因子分组后应用函数

示例：

```
m <- matrix(1:9, nrow = 3)
apply(m, 1, sum)  # 按行求和 (sum by row)
```

2.6 函数与作用域（Functions and Scope）

函数（function）定义格式：

```
add_numbers <- function(a, b) {
  result <- a + b
  return(result)
}
add_numbers(2, 3)
```

R中的变量作用域遵循**词法作用域（lexical scope）**原则：
内部函数可以引用外部环境变量。

示例：

```
make_adder <- function(n) {
  function(x) x + n
}
add5 <- make_adder(5)
add5(10)  # 输出15
```

2.7 字符处理与字符串操作（String Processing）

基础函数

```
s <- "R Language"
nchar(s)                # 字符数
toupper(s)              # 转大写
tolower(s)              # 转小写
substr(s, 1, 3)         # 截取子串
paste("Data", "1001")   # 拼接
```

使用 stringr 包（stringr package）

```
library(stringr)
str_detect("Data1001", "Data")
str_replace("Data1001", "1001", "2026")
str_split("A,B,C", ",")
```

第3章：R中的数据导入与导出（Data Input and Output in R）

3.1 CSV与文本文件（CSV and Text Files）

读取CSV文件（Read CSV File）

```
data <- read.csv("data/sample.csv", header = TRUE, sep = ",")
```

- `header = TRUE`：第一行是否为列名 (whether the first line contains column names)
- `sep = ","`：分隔符 (delimiter)

写入CSV文件（Write CSV File）

```
write.csv(data, "output/result.csv", row.names = FALSE)
```

- `row.names = FALSE`：不保存行名 (do not write row names)

读取制表符文件（Read TSV File）

```
data <- read.table("data/sample.tsv", header = TRUE, sep = "\t")
```

3.2 Excel文件（Excel Files）

导入Excel（Read Excel）

使用 `readxl` 包（package）：

```
library(readxl)
data <- read_excel("data/sample.xlsx", sheet = "Sheet1")
```

写出Excel（Write Excel）

使用 `writexl` 包：

```
library(writexl)
write_xlsx(data, "output/result.xlsx")
```

3.3 JSON与XML文件（JSON and XML Files）

JSON文件（JSON Files）

使用 `jsonlite` 包：

```
library(jsonlite)
json_data <- fromJSON("data/sample.json")
toJSON(json_data, pretty = TRUE)
```

XML文件（XML Files）

使用 `xml2` 包：

```
library(xml2)
xml_data <- read_xml("data/sample.xml")
xml_text(xml_data)
```

3.4 数据库连接与操作（Database Connections and Operations）

R中可以通过多种包访问数据库，如 `RMySQL`，`RSQLite` 等。

SQLite示例（SQLite Example）

```
library(DBI)
conn <- dbConnect(RSQLite::SQLite(), "data/mydb.sqlite")

# 查看表格 (List tables)
dbListTables(conn)

# 读取数据 (Read from table)
data <- dbReadTable(conn, "students")

# 写入数据 (Write to table)
dbWriteTable(conn, "new_table", data)

# 断开连接 (Disconnect)
dbDisconnect(conn)
```

3.5 API与网页数据（API and Web Data）

使用 `httr` 与 `jsonlite` 调用API

```
library(httr)
library(jsonlite)

response <- GET("https://api.exchangerate-api.com/v4/latest/USD")
data <- content(response, "text")
json_data <- fromJSON(data)
json_data$rates$CNY
```

- `GET()`：向API发送HTTP请求 (send HTTP request)
- `content()`：获取响应内容 (extract response content)
- `fromJSON()`：解析JSON结构 (parse JSON format)

3.6 数据清洗与预处理（Data Cleaning and Preprocessing）

在导入文件后，常需要做数据清洗（data cleaning）：

```
library(dplyr)

clean_data <- data %>%
  filter(!is.na(age)) %>%        # 去除缺失值 (remove NA)
  mutate(
    gender = toupper(gender),    # 转为大写 (convert to upper case)
    age_group = ifelse(age > 30, "Older", "Younger")  # 条件判断 (conditional transformation)
  )
```

3.7 其他常见格式（Other Common Formats）

文件类型 (File Type)	包 (Package)	读取函数 (Read Function)	写出函数 (Write Function)
Stata (.dta)	haven	`read_dta()`	`write_dta()`
SPSS (.sav)	haven	`read_sav()`	`write_sav()`
Parquet	arrow	`read_parquet()`	`write_parquet()`
Feather	feather	`read_feather()`	`write_feather()`
R二进制数据	base	`save()` / `load()`	同上

示例：

```
# 保存和加载RData文件
save(clean_data, file = "output/data_clean.RData")
load("output/data_clean.RData")
```

3.8 编码与本地化（Encoding and Localization）

如果遇到中文乱码问题，可以设置文件编码（encoding）：

```
data <- read.csv("data/chinese.csv", fileEncoding = "UTF-8")
```

检查系统语言与编码：

```
Sys.getlocale()
Sys.setlocale("LC_ALL", "Chinese")
```

第4章：R中的数据操作（Data Manipulation in R）

本章主要介绍R中进行数据操作的核心方法，尤其是基于 `dplyr` **包（package）** 的“语义化管道式（pipeline）”写法，这是现代R语言数据分析的核心工具之一。

4.1 数据操作概述（Overview of Data Manipulation）

数据操作（data manipulation）指对原始数据进行**筛选、排序、汇总、变换、连接**等一系列处理，以使数据更适合分析或建模工作。

R中的主流数据操作方式包括：

方法	说明（Description）
基础R函数（Base R functions）	传统的索引和操作方式
`dplyr` 包	提供简洁的链式语法和统一的函数接口
`data.table` 包	高性能数据操作工具，效率极高

本章以 `dplyr` 为主，兼顾Base R的方法。

4.2 dplyr基本语法结构（Basic Grammar of dplyr）

加载包：

```
library(dplyr)
```

核心函数（Core Functions）：

功能（Functionality）	函数名称（Function Name）	示例（Example）
选择列 (Select columns)	`select()`	`select(df, name, age)`
筛选行 (Filter rows)	`filter()`	`filter(df, age > 25)`
排序 (Arrange by order)	`arrange()`	`arrange(df, desc(age))`
创建新变量 (Create new variable)	`mutate()`	`mutate(df, age2 = age^2)`
汇总统计 (Summarize)	`summarise()`	`summarise(df, mean_age = mean(age))`
分组计算 (Group by columns)	`group_by()`	`group_by(df, gender)`

4.3 管道操作符（Pipe Operator：`%>%`）

`%>%` 称为管道操作符（pipe operator），用来将上一步的结果直接传入下一步函数，使代码结构清晰。

示例：

```
result <- df %>%
  filter(age > 25) %>%
  group_by(gender) %>%
  summarise(mean_age = mean(age))
```

该语句等价于嵌套写法：

```
summarise(group_by(filter(df, age > 25), gender), mean_age = mean(age))
```

4.4 数据选择与筛选（Selecting and Filtering Data）

选择列（Selecting Columns）

```
df %>% select(name, age)
df %>% select(starts_with("col"))   # 选择以“col”开头的列
```

筛选行（Filtering Rows）

```
df %>% filter(age >= 20 & gender == "F")
df %>% filter(!(city %in% c("Beijing", "Shanghai")))
```

逻辑符号（Logical Operators）：

符号	含义（Meaning）
`&`	并且 (and)
`|`	或者 (or)
`!`	非 (not)

4.5 排序与取样（Sorting and Sampling）

排序（Arranging）

```
df %>% arrange(age)
df %>% arrange(desc(salary))
```

随机取样（Sampling）

```
sample_n(df, 10)     # 随机取10行
sample_frac(df, 0.2)  # 抽取20%的样本
```

4.6 创建与转换变量（Creating and Transforming Variables）

使用 `mutate()` 创建新列（new columns）：

```
df %>%
  mutate(
    salary_k = salary / 1000,          # 转换单位（convert unit）
    age_group = ifelse(age > 30, "Old", "Young")
  )
```

只保留部分列：

```
df %>% transmute(name, salary_k = salary / 1000)
```

4.7 分组与汇总（Grouping and Summarizing）

使用 `group_by()` 与 `summarise()`：

```
df %>%
  group_by(department) %>%
  summarise(
    avg_salary = mean(salary, na.rm = TRUE),
    max_age = max(age)
  )
```

同时分组多个变量：

```
df %>%
  group_by(department, gender) %>%
  summarise(count = n())
```

4.8 数据连接与合并（Joining and Merging Data）

`dplyr` 提供多种连接方式，类似SQL中的 `JOIN` 操作。

操作（Operation）	函数	说明（Description）
内连接 (Inner join)	`inner_join()`	仅保留匹配行
左连接 (Left join)	`left_join()`	保留左表全部行
右连接 (Right join)	`right_join()`	保留右表全部行
全连接 (Full join)	`full_join()`	保留全部匹配与不匹配行
绑定行 (Row bind)	`bind_rows()`	按行合并数据
绑定列 (Column bind)	`bind_cols()`	按列合并数据

示例：

```
df1 <- data.frame(id = 1:3, score = c(90, 85, 88))
df2 <- data.frame(id = 2:4, grade = c("A", "B", "A"))

merged <- left_join(df1, df2, by = "id")
```

4.9 宽表与长表转换（Wide and Long Format Transformation）

使用 `tidyr` 包操作数据结构。

加载包：

```
library(tidyr)
```

长表转宽表（long → wide）

```
wide_df <- spread(long_df, key = variable, value = value)
```

宽表转长表（wide → long）

```
long_df <- gather(wide_df, key = "variable", value = "value", column1:column3)
```

或者使用新版语法：

```
pivot_longer(df, cols = starts_with("year"), names_to = "Year", values_to = "Value")
pivot_wider(df, names_from = Category, values_from = Value)
```

4.10 缺失值处理（Handling Missing Values）

查看缺失值（missing values）：

```
is.na(df)
sum(is.na(df))
```

删除缺失值（remove NAs）：

```
df_clean <- na.omit(df)
```

替换缺失值（replace NAs）：

```
df %>%
  mutate(age = ifelse(is.na(age), mean(age, na.rm = TRUE), age))
```

4.11 排列与去重（Sorting and Removing Duplicates）

去重函数：

```
distinct(df, name, .keep_all = TRUE)
```

在 `group_by()` 中结合去重：

```
df %>%
  group_by(city) %>%
  summarise(unique_count = n_distinct(name))
```

好的，以下是 **第5章：数据可视化基础（Fundamentals of Data Visualization in R）** 的完整内容，包含关键术语的中英文对照。

第5章：数据可视化基础（Fundamentals of Data Visualization in R）

R语言以其强大的数据可视化能力闻名，尤其是基于 `ggplot2` **包（package）** 的图形系统。本章将介绍从基础绘图到 `ggplot2` 的核心语法结构。

5.1 R中的数据可视化系统（Visualization Systems in R）

R拥有多种绘图体系：

类型（System Type）	包（Package）	主要特点（Main Features）
基础图形系统（Base Graphics）	base	简单直观、命令式绘图
网格系统（Grid Graphics）	grid	支持复杂排版和面板布局
ggplot2系统（ggplot2 system）	ggplot2	基于语法体系（grammar of graphics），最强大且结构化的系统

5.2 安装与加载ggplot2（Installing and Loading ggplot2）

```
install.packages("ggplot2")
library(ggplot2)
```

`ggplot2` 基于 **语法图形学（Grammar of Graphics）** 理念，其核心思想是将图表分为多个独立组件（components）：

- 数据（data）
- 映射（mapping, aesthetic）
- 几何对象（geometry）
- 统计变换（statistical transformation）
- 坐标系统（coordinate system）
- 主题（theme）

5.3 ggplot基础结构（The Structure of ggplot）

基本语句结构：

```
ggplot(data = dataset, aes(x = var1, y = var2)) +
  geom_point()
```

各部分说明：

部分（Component）	说明（Description）
`ggplot()`	初始化图形对象
`data`	指定数据集
`aes()`	定义变量映射关系（mapping between variables and aesthetics）
`geom_`	添加几何对象（geometric layers）

5.4 常用几何对象（Common Geometrical Layers）

函数	图形类型（Chart Type）	示例（Example）
`geom_point()`	散点图（scatter plot）	关系分析
`geom_line()`	折线图（line chart）	时间序列
`geom_bar()`	条形图（bar chart）	分类统计
`geom_histogram()`	直方图（histogram）	分布可视化
`geom_boxplot()`	箱线图（box plot）	离散统计分布
`geom_density()`	核密度图（density plot）	连续变量分布

示例：

```
ggplot(data = mtcars, aes(x = wt, y = mpg)) +
  geom_point(color = "blue", size = 3)
```

5.5 美学映射（Aesthetic Mappings）

`aes()` 设置映射关系，将数据变量映射到视觉特征：

常见美学属性：

属性	英文名称	示例
`x`	x轴变量	`aes(x = height)`
`y`	y轴变量	`aes(y = weight)`
`color`	颜色映射	`aes(color = gender)`
`size`	尺寸映射	`aes(size = population)`
`shape`	点形状	`aes(shape = group)`
`fill`	填充颜色	`aes(fill = category)`

示例：

```
ggplot(mtcars, aes(x = mpg, y = hp, color = factor(cyl))) +
  geom_point()
```

5.6 分面绘图（Faceting）

通过 `facet_wrap()` 和 `facet_grid()` 分面显示（faceting），用于对不同子集单独绘制图表。

```
ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_point() +
  facet_wrap(~ cyl)
```

另一个示例：

```
ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_point() +
  facet_grid(gear ~ cyl)
```

5.7 主题与标签（Themes and Labels）

添加标题与标签（Titles and Labels）

```
ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_point() +
  labs(
    title = "Weight vs. Fuel Efficiency",
    x = "Weight (1000 lbs)",
    y = "Miles per Gallon",
    color = "Cylinders"
  )
```

主题风格（Themes）

`ggplot2` 提供多种内置主题：

```
+ theme_bw()      # 黑白主题
+ theme_minimal() # 简洁风格
+ theme_classic() # 经典风格
```

自定义主题（custom theme）：

```
+ theme(
    plot.title = element_text(size = 14, face = "bold", hjust = 0.5),
    axis.text = element_text(color = "black", size = 12)
  )
```

5.8 坐标系统与缩放（Coordinate Systems and Scales）

坐标系统（Coordinate Systems）

函数	功能（Functionality）
`coord_flip()`	交换x与y轴
`coord_polar()`	极坐标（polar coordinates）
`coord_cartesian()`	限制显示区域（zoom in）

```
ggplot(mtcars, aes(x = factor(cyl), y = mpg)) +
  geom_boxplot() +
  coord_flip()
```

缩放函数（Scale Functions）

用于调整映射的显示方式：

```
scale_color_manual(values = c("red", "blue", "green"))
scale_y_continuous(limits = c(0, 50))
```

5.9 多层组合绘图（Layered Plots）

可以在同一图中叠加多个几何对象（geometric layers）：

```
ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_point(aes(color = factor(cyl))) +
  geom_smooth(method = "lm", se = FALSE)
```

> 说明：
>
> - `geom_point()` 绘制散点
> - `geom_smooth()` 绘制拟合线（fitted line）

5.10 保存图形（Saving Plots）

使用 `ggsave()` 保存图形文件：

```
ggsave("output/plot.png", width = 6, height = 4, dpi = 300)
```

可保存为 `.png`，`.pdf`，`.jpg`，`.svg` 等格式。

5.11 R基础绘图函数（Base Graphics）

虽然 `ggplot2` 是现代主流工具，但基础R绘图依旧有其价值：

```
plot(x = mtcars$wt, y = mtcars$mpg,
     main = "Base R Plot",
     xlab = "Weight", ylab = "MPG",
     col = "blue", pch = 19)
```

添加图例（legend）：

```
legend("topright", legend = c("Data points"), col = "blue", pch = 19)
```

第6章：数据可视化进阶（Advanced Data Visualization in R）

在掌握了第5章的基础绘图方法后，本章将深入探讨 `ggplot2` 的高级技巧与扩展功能，包括多变量可视化、主题定制、交互式图形和动画等。

6.1 复合图表与多变量可视化（Multivariate and Combined Plots）

在探索多维数据时，可以通过混合不同的几何图层（geometric layers）来表达变量之间的复杂关系。

示例：散点 + 回归线（scatter + regression line）

```
ggplot(mtcars, aes(x = wt, y = mpg, color = factor(cyl))) +
  geom_point(size = 3) +
  geom_smooth(method = "lm", se = FALSE)
```

- `geom_point()`：绘制点（points）
- `geom_smooth()`：添加拟合线（fitted line）
- `method = "lm"`：线性回归（linear regression）

示例：双变量关系 + 分布（joint plot）

使用 `ggExtra` 包：

```
library(ggExtra)

p <- ggplot(mtcars, aes(wt, mpg)) + geom_point()
ggMarginal(p, type = "histogram", fill = "skyblue")
```

- `ggMarginal()`：在图形边缘添加分布直方图（marginal histogram）

6.2 高级配色（Advanced Color Customization）

颜色在数据可视化中至关重要，合理配色可以提升信息传达效果。

使用内置调色板（Built-in Palettes）

```
scale_color_brewer(palette = "Set2")
scale_fill_brewer(palette = "Dark2")
```

使用自定义颜色（Manual Colors）

```
scale_color_manual(values = c("4" = "red", "6" = "blue", "8" = "green"))
```

使用渐变色（Gradient Scales）

```
scale_color_gradient(low = "lightblue", high = "darkblue")
```

6.3 自定义主题与字体（Custom Themes and Fonts）

可以使用 `theme()` 函数全面控制图表样式。

```
ggplot(mtcars, aes(wt, mpg)) +
  geom_point(size = 3, color = "darkorange") +
  labs(title = "Custom Theme Example") +
  theme_minimal() +
  theme(
    plot.title = element_text(size = 16, face = "bold", hjust = 0.5),
    panel.grid.major = element_line(color = "gray80"),
    axis.text = element_text(color = "navy", size = 12)
  )
```

设置字体（fonts）：

```
theme(text = element_text(family = "Arial"))
```

使用扩展包 `ggthemes` 增加预设样式：

```
library(ggthemes)
+ theme_economist()
+ theme_wsj()
+ theme_fivethirtyeight()
```

6.4 多图组合与布局（Combining Multiple Plots）

在数据分析报告中，常需要排列多个子图（subplots）。

使用 `patchwork` 包（最简洁方法）

```
library(patchwork)

p1 <- ggplot(mtcars, aes(wt, mpg)) + geom_point()
p2 <- ggplot(mtcars, aes(factor(cyl), mpg)) + geom_boxplot()
p3 <- ggplot(mtcars, aes(mpg)) + geom_histogram(binwidth = 5)

(p1 | p2) / p3   # 上面两个并排，下面一个
```

使用 `gridExtra` 包

```
library(gridExtra)
grid.arrange(p1, p2, p3, ncol = 2)
```

6.5 动态与交互式图表（Interactive and Animated Visualizations）

交互式图表（Interactive Plots）

使用 `plotly` 将 `ggplot2` 图表转换为交互式（interactive）版本：

```
library(plotly)
p <- ggplot(mtcars, aes(x = wt, y = mpg, color = factor(cyl))) +
  geom_point()
ggplotly(p)
```

> 鼠标悬停显示数据、缩放、拖动等交互功能。

动态动画（Animated Plots）

使用 `gganimate` 包：

```
library(gganimate)

p <- ggplot(gapminder, aes(gdpPercap, lifeExp, size = pop, color = continent)) +
  geom_point(alpha = 0.7) +
  scale_x_log10() +
  transition_time(year) +
  labs(title = 'Year: {frame_time}')

animate(p, duration = 10, fps = 20, width = 600, height = 400)
```

输出GIF动画：

```
anim_save("output/gapminder.gif")
```

6.6 地理空间数据可视化（Geospatial Visualization）

使用maps包进行基础绘制

```
library(maps)
map("world")
```

使用ggplot2 + 地图数据

```
library(ggplot2)
library(mapdata)

world <- map_data("world")

ggplot(world, aes(long, lat, group = group)) +
  geom_polygon(fill = "lightgray", color = "white")
```

绘制国家或区域数据

```
ggplot(world, aes(long, lat, group = group, fill = region)) +
  geom_polygon(color = "white") +
  theme_void() +
  theme(legend.position = "none")
```

6.7 极坐标与雷达图（Polar and Radar Plots）

极坐标条形图（Circular Bar Chart）

```
ggplot(mtcars, aes(x = factor(cyl), fill = factor(cyl))) +
  geom_bar() +
  coord_polar()
```

雷达图（Radar Chart）

使用 `fmsb` 包:

```
library(fmsb)

data <- as.data.frame(matrix(sample(1:10, 10), ncol = 5))
colnames(data) <- c("A", "B", "C", "D", "E")

data <- rbind(rep(10,5), rep(0,5), data)
radarchart(data)
```

6.8 高级注释（Annotations）

添加文本、标签或线条说明：

```
ggplot(mtcars, aes(wt, mpg)) +
  geom_point() +
  annotate("text", x = 5, y = 30, label = "High Efficiency", size = 5, color = "red") +
  annotate("segment", x = 5, xend = 4, y = 30, yend = 25, color = "red", arrow = arrow())
```

6.9 高级统计图形（Advanced Statistical Visualizations）

包含分布、相关性和小提琴图等。

图形类型（Chart Type）	函数	描述（Description）
小提琴图（Violin plot）	`geom_violin()`	结合箱线图与密度图
误差线图（Error bars）	`geom_errorbar()`	显示置信区间
相关性矩阵（Correlation matrix）	`corrplot()`	变量关系展示

示例：

```
ggplot(mtcars, aes(factor(cyl), mpg)) +
  geom_violin(fill = "lightblue") +
  geom_boxplot(width = 0.1)
```

6.10 高级案例：综合绘图（Integrated Example）

```
library(ggplot2)
library(patchwork)

p1 <- ggplot(mtcars, aes(x = wt, y = mpg, color = factor(cyl))) +
  geom_point(size = 3) +
  geom_smooth(method = "lm", se = FALSE)

p2 <- ggplot(mtcars, aes(x = factor(cyl), y = mpg, fill = factor(cyl))) +
  geom_boxplot()

p3 <- ggplot(mtcars, aes(x = mpg)) +
  geom_histogram(binwidth = 3, fill = "steelblue", color = "white")

(p1 | p2) / p3 + plot_annotation(title = "Comprehensive Data Visualization in R")
```

非常好 ✅
那我们现在开始完整撰写 **第7A：实验设计与数据分析在 R 中的实现。**
我将严格按你给出的结构（7A.1–7A.6）展开，每节都包括：

- 🧠 核心概念（中英对照）
- 💻 实际 R 代码示例
- 📊 输出与解释
- 🧩 小结

第7A：实验设计与数据分析在 R 中的实现

(Experimental Design and Data Analysis in R)

7A.1 实验设计的基本概念：控制、随机化、重复

(Basic Concepts of Experimental Design: Control, Randomization, Replication)

核心概念

实验设计（**Experimental Design**）的三大原则：

原则	英文术语	含义
控制	Control	控制非实验因素，保持条件一致
随机化	Randomization	将实验单位随机分配到不同组，避免偏倚
重复	Replication	重复实验，降低偶然误差

目标：最大限度减少系统误差，使结果更加可靠。

R 代码示例：随机化与重复

```
set.seed(123)

# 模拟 20 位受试者分配到 A / B 两组
subject_id <- 1:20
group <- sample(c("A", "B"), size = 20, replace = TRUE)
experiment <- data.frame(subject_id, group)
head(experiment)
```

输出说明
每位受试者随机分到实验组或对照组。
`set.seed()` 保证结果可重复。

7A.2 使用 R 模拟随机分组与对照试验

(Simulating Random Groups and Controlled Experiments in R)

思想

随机对照试验（Randomized Controlled Trial, **RCT**）是验证因果关系的“金标准”。
基本结构：

- 一组接受干预（Treatment）
- 一组保持对照（Control）

示例：药物对血压影响的模拟

```
set.seed(42)
n <- 100
group <- rep(c("Treatment", "Control"), each = n/2)

# 模拟治疗组平均降低 8mmHg，对照组平均降低 2mmHg
bp_change <- c(rnorm(50, mean = -8, sd = 3),
               rnorm(50, mean = -2, sd = 3))

trial_data <- data.frame(group, bp_change)
head(trial_data)
```

结果可视化：

```
library(ggplot2)
ggplot(trial_data, aes(group, bp_change, fill = group)) +
  geom_boxplot(alpha = 0.6) +
  labs(title = "RCT: Effect of Treatment on Blood Pressure",
       y = "Δ Blood Pressure (mmHg)") +
  theme_minimal()
```

小结

- 通过随机化可以减少混杂变量影响
- 可以用 R 模拟并可视化 RCT 的结果

7A.3 偏倚与混杂变量：图形可视化与关系分析

(Bias and Confounders: Visualization and Relationship Analysis)

核心分析

- **偏倚（Bias）：** 由于非随机样本或测量方式导致系统性误差
- **混杂变量（Confounder）：** 同时影响自变量和因变量的第三个变量

示例：年龄对药效的混杂作用

```
set.seed(111)
trial_data$age <- rnorm(100, mean = 50, sd = 10)
trial_data$response <- with(trial_data,
                            ifelse(group == "Treatment",
                                   -8 + 0.05 * age + rnorm(100, 0, 3),
                                   -2 + 0.05 * age + rnorm(100, 0, 3)))
```

可视化：

```
ggplot(trial_data, aes(age, response, color = group)) +
  geom_point(alpha = 0.8) +
  geom_smooth(method = "lm", se = FALSE) +
  labs(title = "Effect of Age (Confounding Variable)")
```

观察：年龄可能同时影响治疗组与响应结果 → 混杂构成因果误判风险。

7A.4 随机对照试验（RCT）分析与结果比较

(Analyzing RCT Results in R)

t 检验比较实验与对照

```
t.test(response ~ group, data = trial_data)
```

解释：

- 若 P 值 < 0.05：说明两组均值差异显著。
- 可使用置信区间评估效果范围。

可选：平均差异柱状图

```
library(dplyr)
mean_summary <- trial_data %>%
  group_by(group) %>%
  summarise(mean_response = mean(response))

ggplot(mean_summary, aes(group, mean_response, fill = group)) +
  geom_col(alpha = 0.7) +
  geom_text(aes(label = round(mean_response, 1)), vjust = -0.5)
```

7A.5 双盲实验与安慰剂效应的模拟

(Double-Blind and Placebo Simulation in R)

思想

- 双盲：受试者与实验者都不知道分组
- 安慰剂效应（Placebo Effect）：心理因素导致的改善

模拟双盲实验情境

```
set.seed(999)
group <- rep(c("Drug", "Placebo"), each = 50)
blind_effect <- c(rnorm(50, -10, 3), rnorm(50, -5, 3))
observed <- blind_effect + rnorm(100, 0, 2)

data_blind <- data.frame(group, observed)
ggplot(data_blind, aes(group, observed, fill = group)) +
  geom_boxplot(alpha = 0.7) +
  labs(title = "Double-Blind Experiment with Placebo Effect",
       y = "Observed Change")
```

可使用 `t.test()` 判断差异显著性。

7A.6 相关 vs 因果：线性回归与虚拟实验验证

(Correlation vs Causation: Regression and Simulation)

关键概念：

- **相关（Correlation）≠因果（Causation）**
- **虚拟实验（Simulated Experiment）** 可验证回归关系的稳定性

示例：用线性回归验证因果方向

```
model <- lm(response ~ age + group, data = trial_data)
summary(model)
```

观察：

- `group` 的系数反映干预效果
- 控制了 `age` 后，效果更加准确

可可视化拟合值：

```
trial_data$pred <- predict(model)
ggplot(trial_data, aes(pred, response, color = group)) +
  geom_point() +
  geom_smooth(method = "lm", se = FALSE)
```

好的，以下是完整的 **第7B：观察性研究与统计推断的 R 实现** 部分。格式严谨，无表情或修饰符。

第7B 观察性研究与统计推断的 R 实现

(Observational Studies and Statistical Inference in R)

7B.1 实验研究与观察性研究的差异

(Difference between Experimental and Observational Studies)

概念说明

类型	控制方式	特点	典型应用
实验研究 (Experimental Study)	研究者主动干预并随机分组	能确定因果关系	药物试验、心理实验
观察性研究 (Observational Study)	仅观察数据，无实验控制	难以排除混杂效应，只能推断相关	流行病学调查、社会统计分析

观察性研究中的主要挑战是**自选择偏倚（selection bias）**与**混杂变量（confounder）**。
因此，统计控制与匹配方法在这类研究中尤为重要。

7B.2 使用 R 模拟观察性数据（健康研究示例）

(Simulating Observational Data in R)

示例：吸烟与肺功能之间的关系

```
set.seed(100)
n <- 200
age <- rnorm(n, mean = 50, sd = 12)
smoker <- rbinom(n, 1, prob = plogis((age - 40) / 10))
lung_func <- 100 - 0.6 * age - 8 * smoker + rnorm(n, 0, 5)

obs_data <- data.frame(age, smoker = factor(smoker, labels = c("Non-smoker", "Smoker")),
lung_func)
head(obs_data)
```

可视化：

```
library(ggplot2)
ggplot(obs_data, aes(x = age, y = lung_func, color = smoker)) +
  geom_point(alpha = 0.7) +
  geom_smooth(method = "lm", se = FALSE) +
  labs(title = "Observed Relationship: Smoking and Lung Function")
```

说明：吸烟与年龄同时影响肺功能，存在明显混杂。

7B.3 倾向得分匹配 Propensity Score Matching（MatchIt 包）

(Propensity Score Matching in R Using MatchIt)

原理说明

倾向得分（Propensity Score）表示个体被分配到“处理组”的概率。
通过匹配倾向得分相近的个体，我们可以构造出更可比的处理组与对照组，从而模拟随机实验条件。

R 实现步骤

```
library(MatchIt)

# 建立倾向得分模型
ps_model <- matchit(smoker ~ age, data = obs_data, method = "nearest")

# 匹配结果摘要
summary(ps_model)
```

提取匹配后数据：

```
matched_data <- match.data(ps_model)
ggplot(matched_data, aes(age, lung_func, color = smoker)) +
  geom_point(alpha = 0.8) +
  geom_smooth(method = "lm", se = FALSE) +
  labs(title = "Matched Data: Smoking and Lung Function")
```

解释：匹配后年龄分布更加平衡，混杂效应得到控制。

7B.4 因果效应估计：causalimpact 与 mediation 包

(Causal Effect Estimation with causalimpact and mediation Packages)

1. 使用 causalimpact 包评估干预效果

```
# 示例：假设某健康政策在第70天实施
library(CausalImpact)

# 模拟时间序列
set.seed(10)
x <- 1:100
y <- 80 + 0.3 * x + rnorm(100)
y[71:100] <- y[71:100] + 5  # 假设实施后增加

data_ci <- data.frame(y)
impact <- CausalImpact(data_ci, pre.period = c(1, 70), post.period = c(71, 100))
summary(impact)
plot(impact)
```

输出显示干预前后的平均变化与因果影响估计。

---

2. 中介效应分析：mediation 包

```
library(mediation)

# 模拟示例数据
set.seed(55)
treatment <- rbinom(100, 1, 0.5)
mediator <- 0.5 * treatment + rnorm(100)
outcome <- 1.2 * mediator + 0.3 * treatment + rnorm(100)

data_med <- data.frame(treatment, mediator, outcome)

# 建立模型
model_m <- lm(mediator ~ treatment, data = data_med)
model_y <- lm(outcome ~ treatment + mediator, data = data_med)

# 中介分析
med_out <- mediate(model_m, model_y, treat = "treatment", mediator = "mediator", boot =
TRUE)
summary(med_out)
```

解释：输出包含直接效应、间接效应与总效应，可用于解释因果路径。

7B.5 P值与置信区间解释（t.test, prop.test）

(P-values and Confidence Intervals in R)

1. t 检验

```
t.test(lung_func ~ smoker, data = matched_data)
```

输出包括均值差异、置信区间与 P 值。
若 P < 0.05，则两组均值差异具有统计显著性。

---

2. 比例检验

```
prop.test(x = c(30, 45), n = c(100, 120))
```

结果包含比例差异的置信区间与显著性检验结论。

7B.6 案例研究：从观察到因果的推断过程

(Case Study: From Observation to Causal Inference)

案例：运动频率与心脏健康

步骤说明：

1. 收集观察性数据（运动习惯、年龄、血压指标）。
2. 发现运动频率与血压存在负相关关系。
3. 年龄是潜在混杂变量。
4. 利用倾向得分匹配控制年龄。
5. 用回归模型估计运动对血压的直接影响。

示例模拟：

```
set.seed(101)
n <- 300
age <- rnorm(n, 50, 12)
exercise <- rbinom(n, 1, plogis((60 - age)/10))
bp <- 140 - 5 * exercise + 0.5 * age + rnorm(n, 0, 5)

data_case <- data.frame(age, exercise = factor(exercise, labels = c("Low", "High")), bp)

# 匹配与回归
library(MatchIt)
m2 <- matchit(exercise ~ age, data = data_case, method = "nearest")
matched_data_case <- match.data(m2)

summary(lm(bp ~ exercise, data = matched_data_case))
```

分析结果说明：在控制年龄后，运动频率对血压的降低效果仍然显著，说明存在潜在因果联系。

第7C 概率分布与统计推断在 R 中的计算与可视化

(Probability Distributions and Statistical Inference in R)

7C.1 概率分布的基本概念与分类

(Basic Concepts and Classes of Probability Distributions)

概念说明

概率分布 (Probability Distribution) 用于描述随机变量可能取值及其发生概率。分为：

类型	英文名称	特征	常见分布
离散分布	Discrete	取值为离散数	二项分布 (Binomial), 泊松分布 (Poisson)
连续分布	Continuous	取值为实数区间	正态分布 (Normal), 指数分布 (Exponential)

概率密度函数 (PDF) 与累积分布函数 (CDF) 是分布的基本形式。

7C.2 常用概率分布的 R 实现

(Common Probability Distributions in R)

1. 正态分布

```
x <- seq(-4, 4, by = 0.1)
y <- dnorm(x, mean = 0, sd = 1)
plot(x, y, type = "l", main = "Standard Normal Distribution", ylab = "Density")
```

生成服从指定正态分布的样本：

```
data <- rnorm(1000, mean = 5, sd = 2)
hist(data, breaks = 30, probability = TRUE, col = "lightblue")
lines(density(data), col = "red", lwd = 2)
```

---

2. 二项分布

```
x <- 0:10
prob <- dbinom(x, size = 10, prob = 0.3)
barplot(prob, names.arg = x, main = "Binomial Distribution (n=10, p=0.3)")
```

计算二项分布的累积分布与分位数：

```
pbinom(3, size = 10, prob = 0.3)
qbinom(0.95, size = 10, prob = 0.3)
```

3. 泊松分布

```
x <- 0:15
prob <- dpois(x, lambda = 4)
barplot(prob, names.arg = x, main = "Poisson Distribution (λ = 4)")
```

应用场景：单位时间内事件发生次数（如来电数量、事故次数）。

---

4. 指数分布

```
x <- seq(0, 10, by = 0.1)
y <- dexp(x, rate = 0.5)
plot(x, y, type = "l", main = "Exponential Distribution (λ = 0.5)", ylab = "Density")
```

应用：描述事件之间的时间间隔（如产品寿命）。

7C.3 抽样与中心极限定理示例

(Sampling and the Central Limit Theorem)

示例：验证中心极限定理

```
set.seed(123)
sample_means <- replicate(1000, mean(rexp(40, rate = 1)))
hist(sample_means, breaks = 30, probability = TRUE,
     main = "Sample Means from Exponential Distribution")
lines(density(sample_means), col = "red", lwd = 2)
```

结果说明：原始数据服从指数分布，但样本均值的分布接近正态分布，这验证了中心极限定理。

7C.4 参数估计与最大似然法

(Parameter Estimation and Maximum Likelihood Methods)

1. 使用均值与方差估计正态分布参数

```
set.seed(777)
data <- rnorm(100, mean = 10, sd = 3)
mean(data)
sd(data)
```

2. 使用 fitdistr 函数进行最大似然估计 (MASS 包)

```
library(MASS)
fit <- fitdistr(data, "normal")
fit
```

输出包括估计的均值与标准差以及其标准误差。

7C.5 假设检验与置信区间计算

(Hypothesis Testing and Confidence Intervals)

一样本 t 检验

```
set.seed(202)
sample_data <- rnorm(30, mean = 100, sd = 15)
t.test(sample_data, mu = 95)
```

输出包括均值估计、95% 置信区间及 p 值。

两样本 t 检验

```
group1 <- rnorm(25, 50, 10)
group2 <- rnorm(25, 55, 10)
t.test(group1, group2)
```

若 P 值小于显著性水平（如 0.05），说明两组均值差异显著。

方差分析（ANOVA）

```
group <- rep(c("A", "B", "C"), each = 30)
value <- c(rnorm(30, 5, 1), rnorm(30, 6, 1), rnorm(30, 7, 1))
anova_result <- aov(value ~ group)
summary(anova_result)
```

显著性结果表明各组平均值存在统计差异。

置信区间的计算

```
mean_val <- mean(group1)
sd_val <- sd(group1)
n <- length(group1)
error <- qt(0.975, df = n-1) * sd_val / sqrt(n)
c(mean_val - error, mean_val + error)
```

返回均值的 95% 置信区间范围。

7C.6 概率可视化与蒙特卡洛模拟

(Probability Visualization and Monte Carlo Simulation)

可视化多分布比较

```
x <- seq(-4, 4, by = 0.1)
plot(x, dnorm(x), type = "l", col = "blue", lty = 1, ylim = c(0, 0.5))
lines(x, dt(x, df = 5), col = "red", lty = 2)
lines(x, dlogis(x), col = "green", lty = 3)
legend("topright", legend = c("Normal", "t(5)", "Logistic"),
       col = c("blue", "red", "green"), lty = 1:3)
```

蒙特卡洛估计示例：圆周率 π 的近似计算

```
set.seed(999)
N <- 100000
x <- runif(N, -1, 1)
y <- runif(N, -1, 1)
inside <- (x^2 + y^2) <= 1
pi_estimate <- 4 * mean(inside)
pi_estimate
```

结果显示通过随机抽样可以得到 π 的近似值，该过程体现了概率估计的数值实现。

第8章 高级主题与编程技巧

(Chapter 8: Advanced Topics and Programming Techniques in R)

8.1 R中的面向对象编程（S3, S4, R6）

(Object-Oriented Programming in R: S3, S4, R6)

概念说明

R支持多种面向对象编程（OOP）范式，用于构建可扩展的模块化系统。主要模型包括：

系统	描述	优点	典型应用
S3	轻量级、基于约定的系统	简单灵活	基础R函数系统，如 `print()`，`summary()`
S4	严格类型检查与方法签名	稳定、安全	Bioconductor等科学计算包
R6	现代面向对象模型，支持引用语义	性能与封装性强	大型系统开发、对象持久化

8.1.1 S3系统

定义简单类与方法：

```
person <- function(name, age) {
  structure(list(name = name, age = age), class = "person")
}

print.person <- function(x) {
  cat("Name:", x$name, "| Age:", x$age, "\n")
}

p1 <- person("Alice", 25)
print(p1)
```

解释：
S3 通过 `class` 属性标记对象，函数调度依据 `UseMethod()`。

8.1.2 S4系统

定义类与泛型方法：

```
setClass("Student",
         slots = list(name = "character", score = "numeric"))

setGeneric("describe", function(object) standardGeneric("describe"))

setMethod("describe", "Student",
          function(object) {
            paste(object@name, "scored", object@score)
          })

s1 <- new("Student", name = "Li Ming", score = 92)
describe(s1)
```

解释：
S4支持类与方法的显式定义与类型检查，更适用于大型工程。

8.1.3 R6系统

定义类、字段与方法：

```
library(R6)

Person <- R6Class("Person",
  public = list(
    name = NULL,
    initialize = function(name) { self$name <- name },
    greet = function() {
```

```
      cat("Hello, my name is", self$name, "\n")
    }
  )
)

p <- Person$new("John")
p$greet()
```

解释:
R6 提供与Python或C++相近的引用语义（reference semantics），可直接修改实例属性。

8.2 函数式编程与 purrr

(Functional Programming and the purrr Package)

概念简介

函数式编程（Functional Programming，FP）强调函数作为“第一类对象”，鼓励使用高阶函数（higher-order functions）与无副作用（side-effect-free）的操作。

使用 `purrr` 处理列表与向量

```
library(purrr)

nums <- list(1, 2, 3, 4)
squares <- map(nums, ~ .x^2)
reduce(nums, `+`)
```

核心函数:

函数	含义	示例
`map()`	元素映射	`map_dbl(x, sqrt)`
`reduce()`	聚合归约	`reduce(1:4,*)`
`map2()`	双变量映射	`map2(a, b, sum)`
`pmap()`	多参数映射	`pmap(list(a=a,b=b,c=c), sum)`

8.3 向量化计算与性能优化

(Vectorization and Performance Optimization in R)

8.3.1 向量化计算原理

R 自动对数据结构进行**广播操作（vector recycling）**，避免显式循环，提高计算效率。

```
x <- 1:5
y <- 2:6
x + y
```

与传统 for 循环相比，向量化操作速度显著更高。

8.3.2 data.table 的高效数据处理

```
library(data.table)
dt <- data.table(id = 1:5, score = c(80, 90, 75, 88, 92))
dt[, .(mean_score = mean(score)), by = id %% 2]
```

性能优势：

- 内存就地修改（in-place update）
- 分组计算高速化
- 类SQL语法结构

8.3.3 并行计算 parallel 包

```
library(parallel)
cl <- makeCluster(2)
parSapply(cl, 1:5, function(x) x^2)
stopCluster(cl)
```

目标： 使用多核计算以加速大规模数据任务。

8.4 自定义包开发与文档自动生成 (roxygen2)

(Custom R Package Development and roxygen2 Documentation)

8.4.1 包结构基础

典型R包包含：

```
mypkg/
├─ R/
├─ man/
├─ DESCRIPTION
├─ NAMESPACE
└─ tests/
```

8.4.2 使用 `roxygen2` 自动生成文档

```
#' Add two numbers
#' @param x numeric
#' @param y numeric
#' @return Sum of x and y
#' @export
add <- function(x, y) x + y
```

在项目根目录执行：

```
devtools::document()
```

生成 `man/add.Rd` 的帮助文件，自动与函数同步。

8.5 R中的单元测试 (testthat)

(Unit Testing in R with testthat Package)

核心思想

单元测试（Unit Testing）用于验证每个函数在不同条件下的正确性与稳定性。

```
library(testthat)

test_that("addition works", {
  expect_equal(add(1, 2), 3)
  expect_true(is.numeric(add(2, 3)))
})
```

结合 `devtools::test()` 可批量执行包测试。

8.6 高性能统计计算与C++接口 (Rcpp)

(High-Performance Computing and C++ Integration with Rcpp)

Rcpp 简介

Rcpp 提供与C++的高效接口，使R能直接调用C++代码。

```
library(Rcpp)

cppFunction('double addC(double x, double y) {
  return x + y;
}')

addC(2, 3)
```

结果：

```
[1] 5
```

优势总结

特性	描述
高速执行	比纯R代码快数倍
无缝集成	可直接被R函数调用
可扩展性	支持复杂算法、矩阵计算

第9章 机器学习与预测模型

(Chapter 9: Machine Learning and Predictive Modeling in R)

9.1 监督学习与非监督学习概述

(Overview of Supervised and Unsupervised Learning)

概念说明 (Concepts)

类型	英文名称	目标	常见算法
监督学习	Supervised Learning	学习输入与输出之间关系，用于预测	线性回归、逻辑回归、决策树、随机森林
非监督学习	Unsupervised Learning	在无标签数据中发现结构与模式	聚类分析、主成分分析 (PCA)

例子说明

- 监督学习：根据历史销量预测未来销售额。
- 非监督学习：将用户根据消费特征进行聚类。

9.2 回归模型 (lm, glm, caret)

(Regression Models in R)

9.2.1 线性回归模型 linear model (`lm()`)

```
data(mtcars)
model <- lm(mpg ~ wt + hp, data = mtcars)
summary(model)
```

模型输出解读

- **系数 (Coefficients)** 表示自变量对目标变量的线性影响。
- **R^2** 衡量模型解释变量的能力。

预测示例：

```
new_data <- data.frame(wt = c(2.5, 3.5), hp = c(100, 150))
predict(model, newdata = new_data)
```

9.2.2 广义线性模型 generalized linear model (`glm()`)

```
data("mtcars")
glm_model <- glm(vs ~ mpg + wt, data = mtcars, family = binomial)
summary(glm_model)
```

说明

- `family = binomial` 表示逻辑回归模型 (Logistic Regression)。
- 结果通过 `exp(coef(glm_model))` 可转化为比值比 (Odds Ratio)。

9.2.3 使用 caret 框架统一建模接口

(Unified Modeling Interface with caret)

```
library(caret)
set.seed(123)
train_control <- trainControl(method = "cv", number = 5)
model_caret <- train(mpg ~ wt + hp, data = mtcars,
                     method = "lm", trControl = train_control)
print(model_caret)
```

caret 提供一致的接口、交叉验证及自动调参机制。

9.3 分类模型 (逻辑回归、决策树、随机森林)

(Classification Models: Logistic Regression, Decision Tree, Random Forest)

9.3.1 逻辑回归 (Logistic Regression)

```
data("iris")
iris_data <- subset(iris, Species != "setosa")
iris_data$Species <- factor(iris_data$Species)
log_model <- glm(Species ~ Sepal.Length + Petal.Length, data = iris_data, family = binomial)
summary(log_model)
```

预测：

```
predict(log_model, type = "response")
```

9.3.2 决策树 (Decision Tree)

```
library(rpart)
tree_model <- rpart(Species ~ ., data = iris)
plot(tree_model)
text(tree_model)
```

优点: 模型可解释性强，适用于非线性关系。

9.3.3 随机森林 (Random Forest)

```
library(randomForest)
rf_model <- randomForest(Species ~ ., data = iris, ntree = 200)
print(rf_model)
importance(rf_model)
```

说明:
随机森林通过多棵树的集成提高预测准确性并减少过拟合。

9.4 聚类分析 (k-means, 层次聚类)

(Cluster Analysis)

9.4.1 K-means 聚类

```
set.seed(123)
data <- iris[, 1:4]
km <- kmeans(data, centers = 3, nstart = 20)
table(km$cluster, iris$Species)
```

可视化结果:

```
library(ggplot2)
ggplot(iris, aes(x = Sepal.Length, y = Sepal.Width, color = as.factor(km$cluster))) +
  geom_point(size = 2)
```

9.4.2 层次聚类 (Hierarchical Clustering)

```
dist_matrix <- dist(scale(iris[, 1:4]))
hc <- hclust(dist_matrix, method = "complete")
plot(hc, labels = FALSE)
rect.hclust(hc, k = 3, border = "red")
```

区别: 层次聚类不需要预先设定簇数，可生成树状结构 (dendrogram)。

9.5 模型评估与交叉验证

(Model Evaluation and Cross-Validation)

9.5.1 评估指标 (Evaluation Metrics)

任务类型	常用指标 (Metrics)
回归	RMSE, MAE, R^2
分类	Accuracy, Precision, Recall, F1-score

举例:

```
pred <- predict(log_model, type = "response")
actual <- iris_data$Species
confusionMatrix(as.factor(pred > 0.5), actual)
```

9.5.2 交叉验证 Cross-Validation

```
set.seed(321)
train_control <- trainControl(method = "cv", number = 10)
model_cv <- train(mpg ~ wt + hp, data = mtcars,
                  method = "lm", trControl = train_control)
model_cv$results
```

意义: 通过多次分割训练与验证集估计模型泛化误差。

9.6 使用 tidymodels 框架统一建模流程

(Unified Modeling Workflow with tidymodels)

9.6.1 建模工作流 (Modeling Workflow)

```
library(tidymodels)
data(mtcars)

# 定义模型与预处理配方
lm_spec <- linear_reg() %>%
  set_engine("lm")
lm_recipe <- recipe(mpg ~ wt + hp, data = mtcars) %>%
  step_center(all_predictors()) %>%
  step_scale(all_predictors())

# 建立workflow
lm_workflow <- workflow() %>%
  add_model(lm_spec) %>%
  add_recipe(lm_recipe)

# 拟合模型
lm_fit <- lm_workflow %>%
  fit(data = mtcars)
lm_fit
```

9.6.2 模型调参与评估

```
set.seed(123)
cv_splits <- vfold_cv(mtcars, v = 5)
fit_res <- fit_resamples(lm_workflow, resamples = cv_splits)
collect_metrics(fit_res)
```

tidymodels 为不同模型（如回归、分类、树模型）提供统一接口与结果格式。

第10章 时间序列与金融数据分析

(Chapter 10: Time Series and Financial Data Analysis in R)

10.1 时间序列基础与 ts 对象

(Foundations of Time Series and the `ts` Object)

概念说明 (Concept Overview)

时间序列（Time Series） 是在**按时间顺序（chronological order）**收集的数据点序列，用于描述变量随时间的动态变化趋势。

核心术语	英文	含义
周期 (Frequency)	Frequency	每单位时间内的观测点数量，如季节性数据

核心术语	英文	含义
平稳性 (Stationarity)	Stationarity	统计特征（均值、方差）随时间不变
趋势 (Trend)	Trend	数据随时间的长期变化方向
季节性 (Seasonality)	Seasonality	周期性变化模式

10.1.1 创建与可视化 ts 对象

```
ts_data <- ts(c(100, 120, 130, 150, 160, 170),
              start = c(2020, 1), frequency = 4)
plot(ts_data, main = "Quarterly Sales", ylab = "Value")
```

说明:

- `start = c(2020,1)` 表示起始时间为2020年第一季度。
- `frequency = 4` 表示季度数据（每年4个观测点）。

10.1.2 时间序列基础操作

```
start(ts_data)
end(ts_data)
frequency(ts_data)
window(ts_data, start = c(2020, 2), end = c(2021, 1))
```

可用 `diff()` 计算差分 (difference)，用于去除趋势：

```
diff(ts_data)
```

10.2 ARIMA模型与预测 (forecast 包)

(ARIMA Models and Forecasting with the `forecast` Package)

10.2.1 ARIMA 模型简介

ARIMA (AutoRegressive Integrated Moving Average)
通过参数 (p, d, q) 描述：

- **p**：自回归阶数（autoregressive order）
- **d**：差分阶数（difference order）
- **q**：移动平均阶数（moving average order）

10.2.2 模型拟合与预测

```
library(forecast)
data <- AirPassengers
ts_train <- window(data, end = c(1958, 12))
ts_test <- window(data, start = c(1959, 1))

fit <- auto.arima(ts_train)
summary(fit)

fc <- forecast(fit, h = 12)
plot(fc)
```

说明：

- `auto.arima()` 自动选择最优参数 (p, d, q)。
- `forecast()` 用于生成未来预测值。

10.2.3 模型评估

```
accuracy(fc, ts_test)
```

返回常用指标：

- MAE（Mean Absolute Error）
- RMSE（Root Mean Square Error）
- MAPE（Mean Absolute Percentage Error）

10.3 季节分解与趋势分析

(Seasonal Decomposition and Trend Analysis)

10.3.1 分解模型

```
ts_data <- AirPassengers
decomp <- decompose(ts_data)
plot(decomp)
```

输出包括：

- `trend` （趋势成分）
- `seasonal` （季节成分）
- `random` （残差成分）

10.3.2 使用 stl() 高级分解

```
stl_decomp <- stl(log(AirPassengers), s.window = "periodic")
plot(stl_decomp)
```

`stl()` 基于局部加权拟合 (LOESS) 实现更灵活的趋势提取。

10.3.3 趋势建模与预测

```
trend_values <- stl_decomp$time.series[, "trend"]
plot(trend_values, main = "Extracted Trend Component")
```

趋势项可单独用于长期变化分析与外推预测。

10.4 金融时间序列回报率分析

(Financial Time Series and Return Analysis)

10.4.1 导入与可视化金融数据

```
library(quantmod)
getSymbols("AAPL", from = "2020-01-01", to = "2021-01-01")
chartSeries(AAPL)
```

说明:
`quantmod` 自动从金融数据源（如Yahoo Finance）导入时间序列对象 (`xts` 格式)。

10.4.2 计算收益率 (Return Calculation)

```
library(PerformanceAnalytics)
ret <- dailyReturn(AAPL)
chart.Histogram(ret, main = "Daily Returns Distribution")
mean(ret); sd(ret)
```

概念	公式	说明
简单收益率 (Simple Return)	$r_t = \frac{P_t - P_{t-1}}{P_{t-1}}$	适用于短期变化
对数收益率 (Log Return)	$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right)$	常用于连续复利分析

10.4.3 移动均线与波动性

```
chartSeries(AAPL, TA = "addSMA(20); addSMA(50)")
addBBands()
```

解释:

- SMA：简单移动平均线（Simple Moving Average）
- Bollinger Bands：波动区间指标

10.5 波动率建模与风险度量 (rugarch)

(Volatility Modeling and Risk Measurement with rugarch)

10.5.1 GARCH 模型简介

GARCH（Generalized Autoregressive Conditional Heteroskedasticity）用于描述时间序列波动率的动态变化，特别应用于金融收益序列。

模型形式：

$$\sigma_t^2 = \alpha_0 + \alpha_1 \varepsilon_{t-1}^2 + \beta_1 \sigma_{t-1}^2$$

10.5.2 使用 rugarch 拟合 GARCH 模型

```
library(rugarch)

spec <- ugarchspec(variance.model = list(model = "sGARCH", garchOrder = c(1, 1)),
                   mean.model = list(armaOrder = c(1, 0)),
                   distribution.model = "norm")

fit <- ugarchfit(spec, data = as.numeric(ret))
show(fit)
```

10.5.3 波动率预测

```
forecast_vol <- ugarchforecast(fit, n.ahead = 10)
plot(forecast_vol, which = 3)
```

输出图展示未来10期的波动率估计路径。

10.5.4 风险度量指标 (Risk Metrics)

```
VaR <- quantile(ret, probs = 0.05, na.rm = TRUE)
ES  <- mean(ret[ret < VaR], na.rm = TRUE)
VaR; ES
```

指标	英文全称	含义
VaR	Value at Risk	在一定置信度下的最大潜在损失
ES	Expected Shortfall	超出VaR部分的期望损失

第11章 文本挖掘与自然语言处理

(Chapter 11: Text Mining and Natural Language Processing in R)

11.1 文本数据结构与清洗

(Text Data Structure and Cleaning)

概念说明 (Concept Overview)

文本挖掘（Text Mining） 是从大量非结构化文本中提取有价值信息的过程。
自然语言处理（NLP, Natural Language Processing） 则是让计算机理解、分析并生成人类语言的技术核心。

步骤 (Step)	英文名称	主要目的
数据获取	Data Acquisition	读取原始文本
预处理	Preprocessing	去除噪声、标准化文本
特征提取	Feature Extraction	转换为可分析的结构化形式
建模与分析	Modeling & Analysis	进行分类、聚类、主题建模等

11.1.1 导入与基本清洗

```
library(tm)

texts <- c("Data Science is amazing!", "R is a great language for analytics.")
corpus <- Corpus(VectorSource(texts))

corpus <- tm_map(corpus, content_transformer(tolower))
corpus <- tm_map(corpus, removePunctuation)
corpus <- tm_map(corpus, removeWords, stopwords("en"))
inspect(corpus)
```

说明：

- `Corpus`：文本集合对象
- `tm_map()`：对语料库进行函数式映射
- `stopwords()`：停用词，如“the”、“and”

11.1.2 分词与词干化 (Tokenization & Stemming)

```
library(SnowballC)
corpus <- tm_map(corpus, stemDocument, language = "en")
inspect(corpus)
```

关键词（Keywords）

- **Tokenization 分词**：将文本拆解为独立词语
- **Stemming 词干化**：还原为词根（如 running → run）

11.2 文本表示模型 (BoW & TF-IDF)

(Text Representation Models: Bag of Words & TF-IDF)

11.2.1 词袋模型 Bag-of-Words (BoW)

```
dtm <- DocumentTermMatrix(corpus)
inspect(dtm[1:2, 1:5])
```

定义：
BoW 将文本表示为**词频矩阵（term frequency matrix）**，丢弃语序信息，仅保留词频统计。

11.2.2 TF-IDF 权重

(Term Frequency–Inverse Document Frequency)

```
dtm_tfidf <- weightTfIdf(dtm)
as.matrix(dtm_tfidf)[1:2, 1:5]
```

概念	公式	含义
词频 (TF)	$TF_{i,j} = \frac{n_{i,j}}{\sum_k n_{k,j}}$	某词在文档 j 中出现比例
逆文档频率 (IDF)	$IDF_i = \log\left(\frac{N}{n_i}\right)$	区分常见与稀有词
TF-IDF	$TF_{i,j} \times IDF_i$	词的重要性权重

11.3 主题建模 (Topic Modeling)

(Latent Semantic and Topic Modeling)

11.3.1 潜在狄利克雷分配 LDA (Latent Dirichlet Allocation)

```
library(topicmodels)
lda_model <- LDA(dtm, k = 2, control = list(seed = 123))
topics(lda_model)
terms(lda_model, 5)
```

解释:

- `k`：主题数量
- `terms()`：每个主题的主要关键词
- LDA 是通过统计方法推测文档与主题的隐含分布。

11.3.2 主题可视化

```
library(LDAvis)
# 需借助 json 格式数据
# createJSON() -> serVis()
```

LDAvis 提供交互式主题结构展示（关键词分布、主题距离等）。

11.4 文本分类 (Text Classification)

(Supervised Text Classification)

11.4.1 朴素贝叶斯分类模型 Naive Bayes

```
library(e1071)
data <- data.frame(
  text = c("R is powerful", "I love Python", "Data science tools"),
  label = c("R", "Python", "General")
)

# 简化演示，实际需特征向量化
classifier <- naiveBayes(label ~ ., data)
```

原理:
朴素贝叶斯假设特征独立，通过最大化后验概率 $P(y|x)$ 进行分类。

11.4.2 结合 tidytext 的文本建模

```
library(tidytext)
library(dplyr)

text_df <- tibble(line = 1:3,
                  text = c("R language is important", "Python is flexible", "Data analysis
rocks"))

token_df <- text_df %>%
  unnest_tokens(word, text) %>%
  anti_join(stop_words)
token_df
```

11.4.3 文本分类实例 (tidymodels)

```
library(tidymodels)

# 训练/测试划分
set.seed(123)
data_split <- initial_split(sentiments, prop = 0.8)
train_data <- training(data_split)
test_data <- testing(data_split)

# 模型定义与训练示意（如逻辑回归）
log_spec <- logistic_reg() %>% set_engine("glm")
```

11.5 情感分析 (Sentiment Analysis)

(Sentiment Analysis in R)

11.5.1 使用 tidytext 与情感词典

```
library(tidytext)
library(textdata)

sentiments <- get_sentiments("bing")
text_data <- tibble(line = 1:2,
                    text = c("I love data", "I hate errors"))

tidy_words <- text_data %>%
  unnest_tokens(word, text) %>%
  inner_join(sentiments, by = "word")
tidy_words
```

说明:

- “bing”、“nrc”、“afinn” 等为常用英文情感词典。

- 匹配后可统计情感倾向（正向/负向词汇比例）。

11.5.2 情感可视化

```
library(ggplot2)

tidy_words %>%
  count(sentiment) %>%
  ggplot(aes(x = sentiment, y = n, fill = sentiment)) +
  geom_col()
```

图形展示文本中积极与消极情绪词分布情况。

11.6 词向量与深度文本表示 (Word Embeddings)

(Word Embeddings and Deep Representations)

11.6.1 词向量概念

每个单词被映射为一个数值向量，以捕捉词义相似度。
常用模型：

- Word2Vec
- GloVe
- FastText

11.6.2 在 R 中使用 text2vec

```
library(text2vec)
tokens <- word_tokenizer(c("machine learning is fun"))
it <- itoken(tokens, progressbar = FALSE)
vocab <- create_vocabulary(it)
vectorizer <- vocab_vectorizer(vocab)

tcm <- create_tcm(it, vectorizer, skip_grams_window = 5)
glove <- GlobalVectors$new(rank = 50, x_max = 10)
word_vectors <- glove$fit_transform(tcm, n_iter = 10)
```

关键词解释：

- Token：单词或子词单元
- Vectorization：将语义转化为数值向量
- TCM（Term Co-occurrence Matrix）：词共现矩阵

11.6.3 相似度与语义计算

```
cos_sim <- sim2(word_vectors, method = "cosine")
cos_sim[1:5, 1:5]
```

利用**余弦相似度（cosine similarity）** 衡量两个词的语义接近程度。

第12章 网络分析与图结构建模

(Chapter 12: Network Analysis and Graph Modeling in R)

12.1 网络数据与图模型基础

(Foundations of Network Data and Graph Models)

基本概念 (Core Concepts)

网络分析 (Network Analysis) 研究对象间的关系结构，如社交网络、交通网络、生物关系网等。
节点 (node / vertex) 表示实体，边 (edge) 表示连接关系。

概念	英文	说明
节点	Node / Vertex	网络中的实体对象
边	Edge	节点间的连接关系
有向图	Directed Graph	方向性连接
无向图	Undirected Graph	相互作用关系
权重	Weight	边的强度或重要性

12.1.1 创建图对象

```
library(igraph)

# 定义边列表
edges <- data.frame(
  from = c("A", "A", "B", "C", "D"),
  to = c("B", "C", "D", "D", "A")
)

g <- graph_from_data_frame(edges, directed = TRUE)
plot(g, vertex.color = "lightblue", vertex.size = 30, edge.arrow.size = .4)
```

说明:
`graph_from_data_frame()` 可直接从数据框生成图结构对象。

12.1.2 邻接矩阵与邻接表

```
get.adjacency(g)
as_edgelist(g)
```

形式	英文名称	特点
邻接矩阵	Adjacency Matrix	用矩阵元素表示链接存在与权重
邻接表	Edge List	边记录形式，适合稀疏网络

12.2 网络描述性统计

(Descriptive Network Statistics)

12.2.1 度分布 (Degree Distribution)

```
degree(g)
hist(degree(g), main = "Degree Distribution")
```

指标	英文名称	含义
入度	In-degree	指向节点的链接数
出度	Out-degree	节点指向外部的链接数
度分布	Degree Distribution	节点连接模式整体特性

12.2.2 中心性指标 (Centrality Metrics)

```
data.frame(
  degree = degree(g),
  betweenness = betweenness(g),
  closeness = closeness(g),
  eigen = evcent(g)$vector
)
```

指标	说明
度中心性 (Degree Centrality)	节点的直接连接数量
中介中心性 (Betweenness)	节点作为“桥梁”的重要程度
近接中心性 (Closeness)	节点到其他节点的平均距离
特征向量中心性 (Eigenvector Centrality)	考虑邻居节点影响力的高层衡量

12.2.3 图密度与聚集系数

```
edge_density(g)
transitivity(g, type = "global")
```

概念	意义
图密度 (Graph Density)	边连接的稠密程度
聚集系数 (Clustering Coefficient)	三角闭合程度（好友-好友关系）

12.3 社区发现与群体结构

(Community Detection and Modular Structure)

12.3.1 Louvain 社区算法

```
community <- cluster_louvain(as.undirected(g))
plot(community, g)
membership(community)
```

说明：
Louvain 方法通过最大化模块度 (modularity) 寻找群体划分，即网络中连结更紧密的子群体。

12.3.2 模块度计算 (Modularity)

```
modularity(community)
```

模块度越高，表示划分后的社区结构越明显。

12.3.3 其他社区检测方法

方法	函数	特点
Walktrap	`cluster_walktrap()`	基于随机游走
Fast Greedy	`cluster_fast_greedy()`	适合稀疏图
Edge Betweenness	`cluster_edge_betweenness()`	基于边分割的重要性

12.4 网络可视化与布局

(Network Visualization and Layouts)

12.4.1 经典布局算法

```
set.seed(123)
plot(g, layout = layout_with_fr, vertex.label.color="black")
```

常见布局：

- **Fruchterman-Reingold (`layout_with_fr`)**：力导向布局
- **Kamada-Kawai (`layout_with_kk`)**：距离最优布局
- **Circle (`layout_in_circle`)**：节点环形排列

12.4.2 可视化美化元素

```
V(g)$color <- degree(g)
V(g)$size <- 10 + 2 * degree(g)
plot(g, vertex.label = V(g)$name,
     vertex.color = heat.colors(length(V(g))),
     edge.arrow.size = 0.3)
```

12.4.3 动态网络与交互式可视化

```
library(visNetwork)

nodes <- data.frame(id = c("A", "B", "C", "D"))
visNetwork(nodes, edges, main = "Interactive Graph")
```

`visNetwork` 可以生成 HTML 交互式网络图，支持动态拖动、放大缩小、节点信息显示。

12.5 加权网络与路径分析

(Weighted Networks and Path Analysis)

12.5.1 加权图构建

```
edges_w <- data.frame(
  from = c("A", "B", "C"),
  to = c("B", "C", "D"),
  weight = c(2, 5, 3)
)
g_w <- graph_from_data_frame(edges_w, directed = FALSE)
E(g_w)$width <- E(g_w)$weight
plot(g_w)
```

12.5.2 最短路径与距离

```
shortest_paths(g, from = "A", to = "D")
distances(g)
```

指标	英文	含义
最短路径 (Shortest Path)	Shortest Path	网络中两节点最小跳数
网络直径 (Diameter)	Diameter	网络中最长最短路径
平均距离 (Average Path Length)	Mean Distance	全局连通度指标

12.5.3 最小生成树 (Minimum Spanning Tree)

```
mst_g <- mst(g_w, weights = E(g_w)$weight)
plot(mst_g)
```

说明：
MST用于构造连通全图的最小代价树结构，在通信或物流网络中广泛应用。

12.6 动态网络与演化分析

(Dynamic and Temporal Networks)

12.6.1 igraph + networkDynamic

```
library(networkDynamic)
g_dyn <- network.initialize(5, directed = TRUE)
add.edges.active(g_dyn, tail = 1:4, head = 2:5, onset = 1, terminus = 10)
timeline(g_dyn)
```

关键思想:
将时间维度（onset/terminus）纳入图结构，分析网络结构随时间的演化。

12.6.2 网络动态可视化

```
library(tsna)
compute.animation(g_dyn)
```

可绘制节点连结随时间变化的动画，用于社交传播、关系演变分析。

太好了 ✅
下面为你输出 **第13章：地理空间与地图可视化（Geospatial and Map Visualization in R）** 的**详细完整版**，延续前两章的结构风格。

第13章 地理空间与地图可视化

(Chapter 13: Geospatial Analysis and Map Visualization in R)

13.1 地理空间数据基础

(Fundamentals of Spatial Data)

13.1.1 空间数据类型

在 R 中，地理数据一般包含几何与属性信息，常以 **矢量 (Vector)** 和 **栅格 (Raster)** 形式表示。

类型	英文名称	内容	常用包
点	Point	经纬度坐标，如城市位置	`sf`, `sp`
线	Line	道路、河流等连续路径	`sf`, `sp`
面	Polygon	区域边界，如省份轮廓	`sf`, `rgeos`
栅格	Raster	规则网格数据，如气温分布	`raster`, `terra`

13.1.2 sf 包：现代地理数据接口

```
library(sf)

# 从 shapefile 读取
nc <- st_read(system.file("shape/nc.shp", package = "sf"), quiet = TRUE)

# 查看几何类型与投影信息
st_geometry_type(nc)
st_crs(nc)
plot(nc["AREA"])
```

说明：

- `sf` 是 “Simple Features” 数据标准，现代 R 空间分析的核心工具包。
- `st_crs()` 访问或修改坐标参考系统（Coordinate Reference System, CRS）。

13.2 投影与坐标系统

(Coordinate Systems and Projections)

13.2.1 坐标系转换示例

```
nc_wgs <- st_transform(nc, crs = 4326)
plot(st_geometry(nc_wgs))
```

常用坐标系	EPSG Code	说明
WGS 84	4326	全球通用，经纬度制
Web Mercator	3857	网络地图常用投影
UTM	326XX / 327XX	精确区域投影，XX 为分区号

13.2.2 坐标距离计算

```
p1 <- st_point(c(116.4, 39.9)) %>% st_sfc(crs = 4326)
p2 <- st_point(c(121.5, 31.2)) %>% st_sfc(crs = 4326)

st_distance(p1, p2)
```

输出: 两地之间的地理距离（单位默认米，基于 CRS）。

13.3 地理空间操作

(Spatial Operations)

13.3.1 缓冲区 (Buffer)

```
buf <- st_buffer(p1, dist = 0.5)
plot(buf, col = "lightblue")
```

> 用于定义某点周围半径范围，例如城市影响圈。

13.3.2 空间叠加与交集

```
library(dplyr)
nc_small <- nc %>% slice(1:10)
nc_clip <- st_intersection(nc, st_union(st_geometry(nc_small)))
plot(nc_clip["AREA"])
```

操作类型	函数	说明
交集	`st_intersection()`	获取重叠范围
并集	`st_union()`	合并多个对象

操作类型	函数	说明
差集	`st_difference()`	取区域差异
包含关系	`st_contains()` / `st_within()`	判断包含或位于内部

13.4 栅格数据分析

(Raster Data Handling)

13.4.1 加载与绘制栅格

```
library(raster)
r <- raster(system.file("external/test.grd", package = "raster"))
plot(r)
```

13.4.2 栅格运算

```
r2 <- r * 2 - 50
plot(r2)
```

常见运算包括：

- **重分类**（`reclassify()`）
- **裁剪**（`crop()`）
- **合并**（`merge()`）
- **插值**（`interpolate()`）

13.5 基本地图绘制

(Basic Map Plotting)

13.5.1 使用 ggplot2 + sf

```
library(ggplot2)

ggplot(data = nc) +
  geom_sf(fill = "lightgreen", color = "grey40") +
  theme_minimal() +
  labs(title = "North Carolina Map (sf + ggplot2)")
```

优势：`geom_sf()` 可自然处理投影与几何类型，可叠加统计结果。

13.5.2 添加城市点与注释

```
cities <- data.frame(
  name = c("Beijing", "Shanghai"),
  lon = c(116.4, 121.5),
  lat = c(39.9, 31.2)
)

ggplot() +
  geom_sf(data = world, fill = "antiquewhite") +
  geom_point(data = cities, aes(x = lon, y = lat), color = "red", size = 3) +
  geom_text(data = cities, aes(x = lon, y = lat, label = name), nudge_y = 1) +
  theme_minimal()
```

13.6 交互式地图可视化

(Interactive Map Visualization)

13.6.1 leaflet 包动态地图

```
library(leaflet)

leaflet() %>%
  addTiles() %>%
  addMarkers(lng = 116.4, lat = 39.9, popup = "Beijing") %>%
  addMarkers(lng = 121.5, lat = 31.2, popup = "Shanghai")
```

说明：

- `addTiles()`：加载默认底图 (OpenStreetMap)。
- `addMarkers()`：添加标记点。
- 交互式操作包括拖动、缩放、弹出框交互。

13.6.2 tmap 包专题地图

```
library(tmap)

tmap_mode("plot")
tm_shape(nc) +
  tm_polygons("AREA", palette = "YlGnBu", title = "County Area") +
  tm_layout(main.title = "North Carolina Area Map")
```

`tmap` 支持两种模式：

- `"plot"`：静态图形（适合出版）
- `"view"`：交互模式，类似 leaflet

13.7 空间统计与热点分析

(Spatial Statistics and Hotspot Detection)

13.7.1 全局自相关 (Moran's I)

```
library(spdep)

# 创建邻接矩阵
nc_nb <- poly2nb(nc)
nc_w <- nb2listw(nc_nb, style = "W")

# 计算全局 Moran's I
moran.test(nc$AREA, nc_w)
```

指标	含义
Moran's I > 0	空间聚集（相似值邻近）
Moran's I < 0	空间离散（差异大）
P 值显著	存在空间自相关结构

13.7.2 局部指标 (LISA — Local Indicators of Spatial Association)

```
local_moran <- localmoran(nc$AREA, nc_w)
nc$Ii <- local_moran[,1]
tm_shape(nc) + tm_fill("Ii", palette = "RdBu", title = "Local Moran's I")
```

用途： 标识空间热点与冷点区域。

13.8 地理空间分析综合案例

(Integrated Spatial Case Study)

案例：城市空气质量分析

数据准备

```
library(readr)
air <- read_csv("city_air.csv")   # 包含城市名、经纬度、PM2.5数据
```

转为 sf 对象并绘图

```
air_sf <- st_as_sf(air, coords = c("lon", "lat"), crs = 4326)
ggplot() +
  geom_sf(data = china_prov_sf, fill = "grey90") +
  geom_sf(data = air_sf, aes(color = PM25, size = PM25)) +
  scale_color_viridis_c(option = "C") +
  theme_minimal()
```

分析与可视化结果

- 高值区集中于华北平原，呈现空间聚集。
- 可进一步结合 `moran.test()` 分析全局自相关。